

AI Assisted Coding

Assignment-12.5

Name: K. Sai Krishna Reddy

Hall Ticket: 2303A52305

Batch: 45

Task Description #1 (Sorting – Merge Sort Implementation)

- Task: Use AI to generate a Python program that implements the Merge Sort algorithm.

- Instructions:

o Prompt AI to create a function `merge_sort(arr)` that

sorts a list in ascending order.

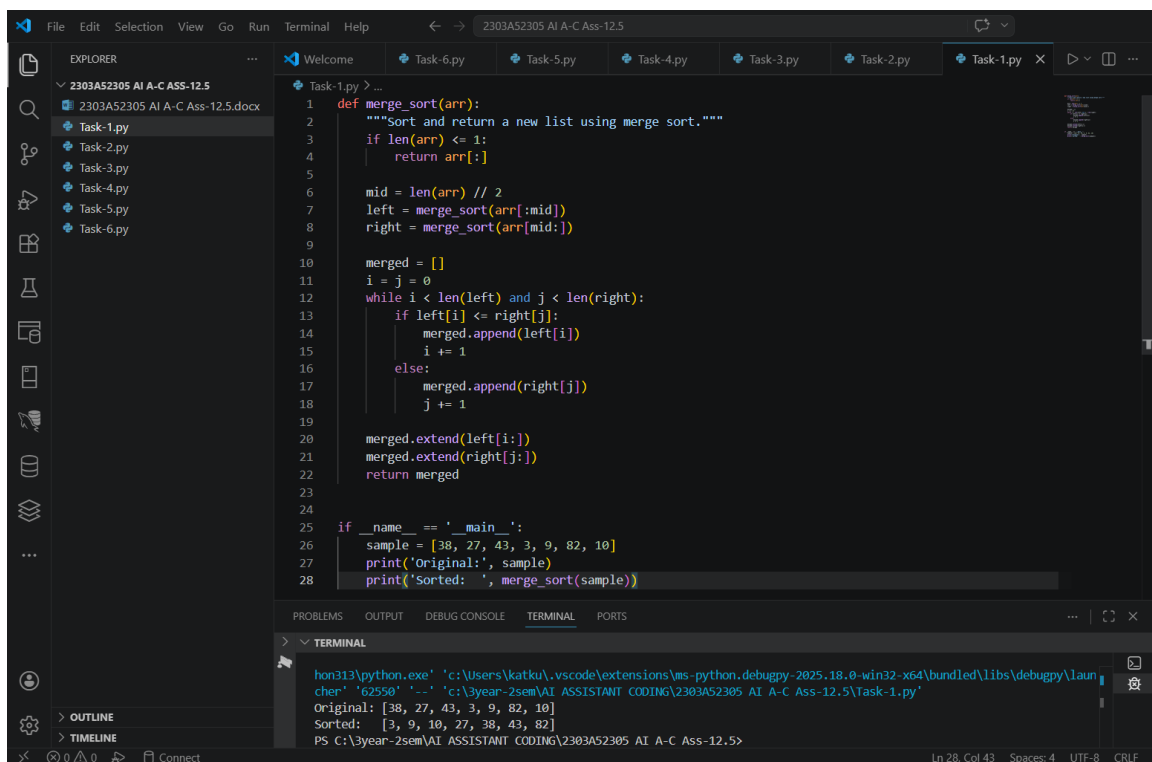
o Ask AI to include time complexity and space

complexity in the function docstring.

o Verify the generated code with test cases.

- Expected Output:

o A functional Python script implementing Merge Sort with proper documentation.



```
File Edit Selection View Go Run Terminal Help
2303A52305 AI A-C Ass-12.5

EXPLORER
2303A52305 AI A-C Ass-12.5
  Task-1.py
  Task-2.py
  Task-3.py
  Task-4.py
  Task-5.py
  Task-6.py

Task-1.py > ...
1 def merge_sort(arr):
2     """Sort and return a new list using merge sort."""
3     if len(arr) <= 1:
4         return arr[:]
5
6     mid = len(arr) // 2
7     left = merge_sort(arr[:mid])
8     right = merge_sort(arr[mid:])
9
10    merged = []
11    i = j = 0
12    while i < len(left) and j < len(right):
13        if left[i] <= right[j]:
14            merged.append(left[i])
15            i += 1
16        else:
17            merged.append(right[j])
18            j += 1
19
20    merged.extend(left[i:])
21    merged.extend(right[j:])
22    return merged
23
24
25 if __name__ == '__main__':
26     sample = [38, 27, 43, 3, 9, 82, 10]
27     print('Original:', sample)
28     print('Sorted: ', merge_sort(sample))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
> TERMINAL
hon313\python.exe 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\laun
cher' '62550' '-' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-12.5\Task-1.py'
Original: [38, 27, 43, 3, 9, 82, 10]
Sorted:  [3, 9, 10, 27, 38, 43, 82]
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-12.5>
```

Task Description #2 (Searching – Binary Search with AI

Optimization)

- Task: Use AI to create a binary search function that finds a target element in a sorted list.

- Instructions:

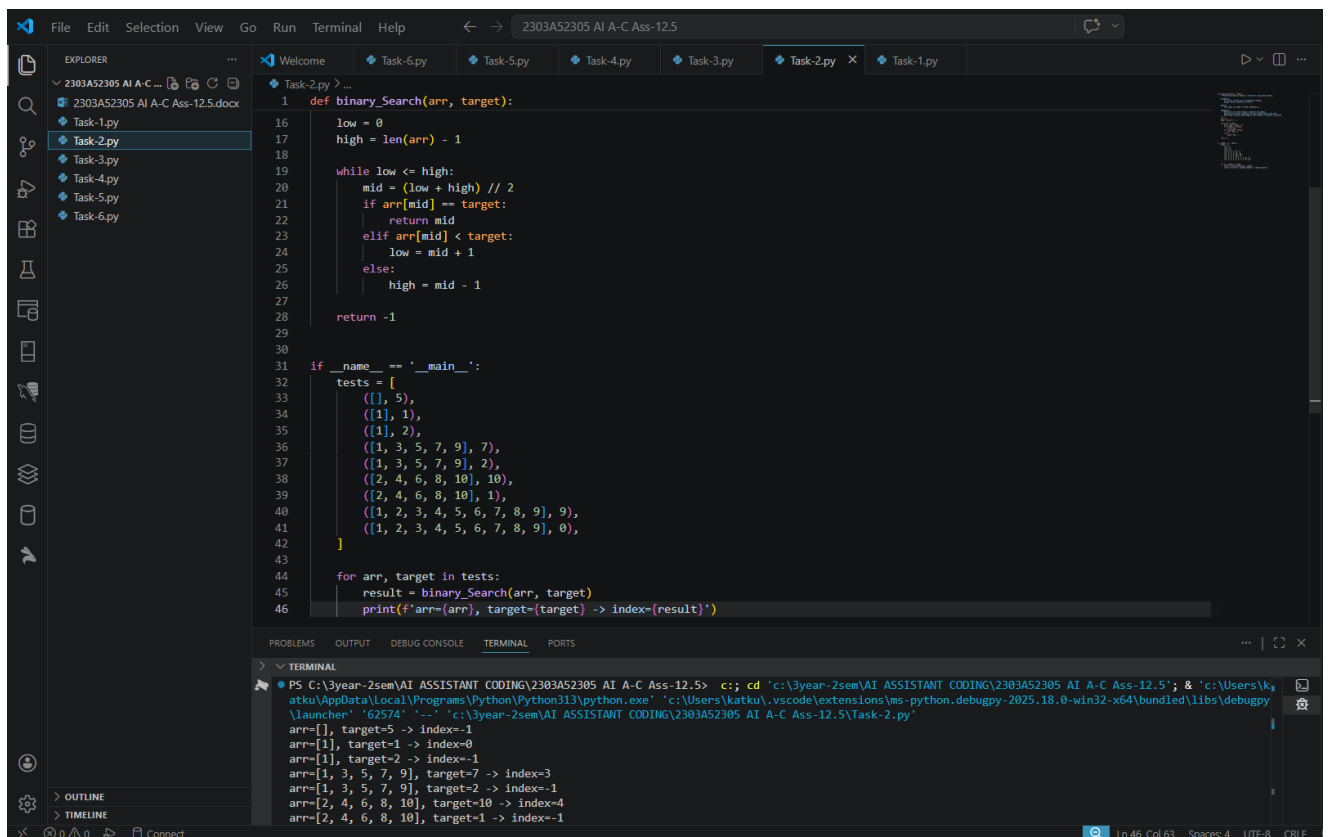
- o Prompt AI to create a function `binary_search(arr, target)` returning the index of the target or -1 if not found.

- o Include docstrings explaining best, average, and worst-case complexities.

- o Test with various inputs.

- Expected Output:

- o Python code implementing binary search with AI-generated comments and docstrings.



```
def binary_search(arr, target):
    """
    Binary search function that finds the index of a target element in a sorted list.
    Returns the index of the target or -1 if not found.
    """
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1

if __name__ == '__main__':
    tests = [
        ([], 5),
        ([1], 1),
        ([1], 2),
        ([1, 3, 5, 7, 9], 7),
        ([1, 3, 5, 7, 9], 2),
        ([2, 4, 6, 8, 10], 10),
        ([2, 4, 6, 8, 10], 1),
        ([1, 2, 3, 4, 5, 6, 7, 8, 9], 9),
        ([1, 2, 3, 4, 5, 6, 7, 8, 9], 0),
    ]

    for arr, target in tests:
        result = binary_search(arr, target)
        print(f'arr={arr}, target={target} -> index={result}')
```

Terminal Output:

```
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-12.5> c:\cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-12.5'; & 'c:\Users\k...
atku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy
\launcher' '625741' '-c' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-12.5\Task-2.py'
arr=[], target=5 -> index=-1
arr=[1], target=2 -> index=-1
arr=[1], target=1 -> index=0
arr=[1, 3, 5, 7, 9], target=7 -> index=3
arr=[1, 3, 5, 7, 9], target=2 -> index=-1
arr=[2, 4, 6, 8, 10], target=10 -> index=4
arr=[2, 4, 6, 8, 10], target=1 -> index=-1
```

Task Description #3: Smart Healthcare Appointment Scheduling

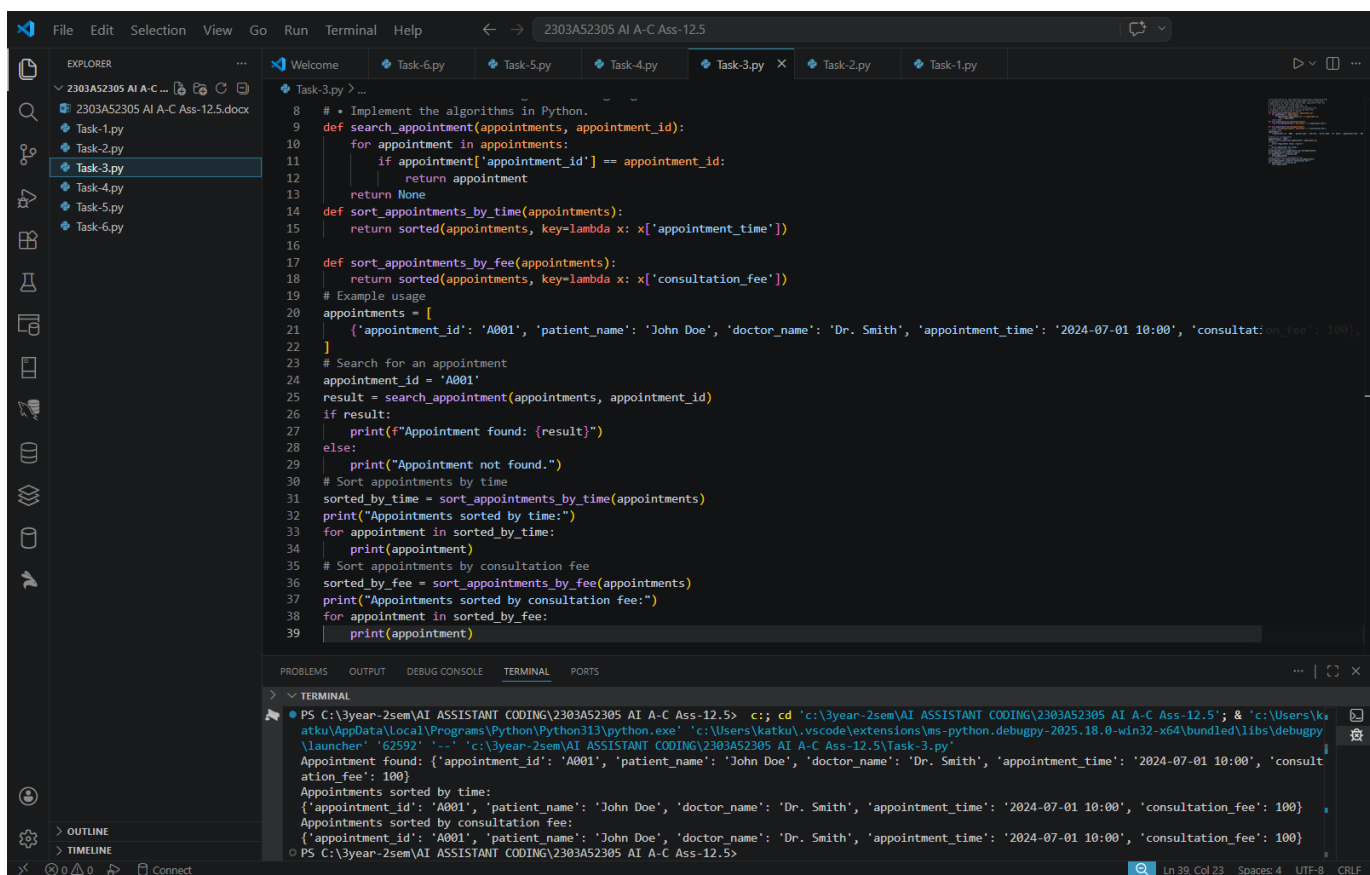
System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.
2. Sort appointments based on time or consultation fee.

Student Task

- Use AI to recommend suitable searching and sorting algorithms.
- Justify the selected algorithms.
- Implement the algorithms in Python.



The screenshot shows a Visual Studio Code editor with a Python file named `Task-3.py` open. The file contains the following code:

```
8 # * Implement the algorithms in Python.
9 def search_appointment(appointments, appointment_id):
10     for appointment in appointments:
11         if appointment['appointment_id'] == appointment_id:
12             return appointment
13     return None
14 def sort_appointments_by_time(appointments):
15     return sorted(appointments, key=lambda x: x['appointment_time'])
16
17 def sort_appointments_by_fee(appointments):
18     return sorted(appointments, key=lambda x: x['consultation_fee'])
19 # Example usage
20 appointments = [
21     {'appointment_id': 'A001', 'patient_name': 'John Doe', 'doctor_name': 'Dr. Smith', 'appointment_time': '2024-07-01 10:00', 'consultation_fee': 100},
22 ]
23 # Search for an appointment
24 appointment_id = 'A001'
25 result = search_appointment(appointments, appointment_id)
26 if result:
27     print(f"Appointment found: {result}")
28 else:
29     print("Appointment not found.")
30 # Sort appointments by time
31 sorted_by_time = sort_appointments_by_time(appointments)
32 print("Appointments sorted by time:")
33 for appointment in sorted_by_time:
34     print(appointment)
35 # Sort appointments by consultation fee
36 sorted_by_fee = sort_appointments_by_fee(appointments)
37 print("Appointments sorted by consultation fee:")
38 for appointment in sorted_by_fee:
39     print(appointment)
```

The terminal output shows the execution of the script:

```
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-12.5> c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe c:\Users\katku\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher '62592' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-12.5\Task-3.py'
Appointment found: {'appointment_id': 'A001', 'patient_name': 'John Doe', 'doctor_name': 'Dr. Smith', 'appointment_time': '2024-07-01 10:00', 'consultation_fee': 100}
Appointments sorted by time:
{'appointment_id': 'A001', 'patient_name': 'John Doe', 'doctor_name': 'Dr. Smith', 'appointment_time': '2024-07-01 10:00', 'consultation_fee': 100}
Appointments sorted by consultation fee:
{'appointment_id': 'A001', 'patient_name': 'John Doe', 'doctor_name': 'Dr. Smith', 'appointment_time': '2024-07-01 10:00', 'consultation_fee': 100}
```

Task Description #4: Railway Ticket Reservation System

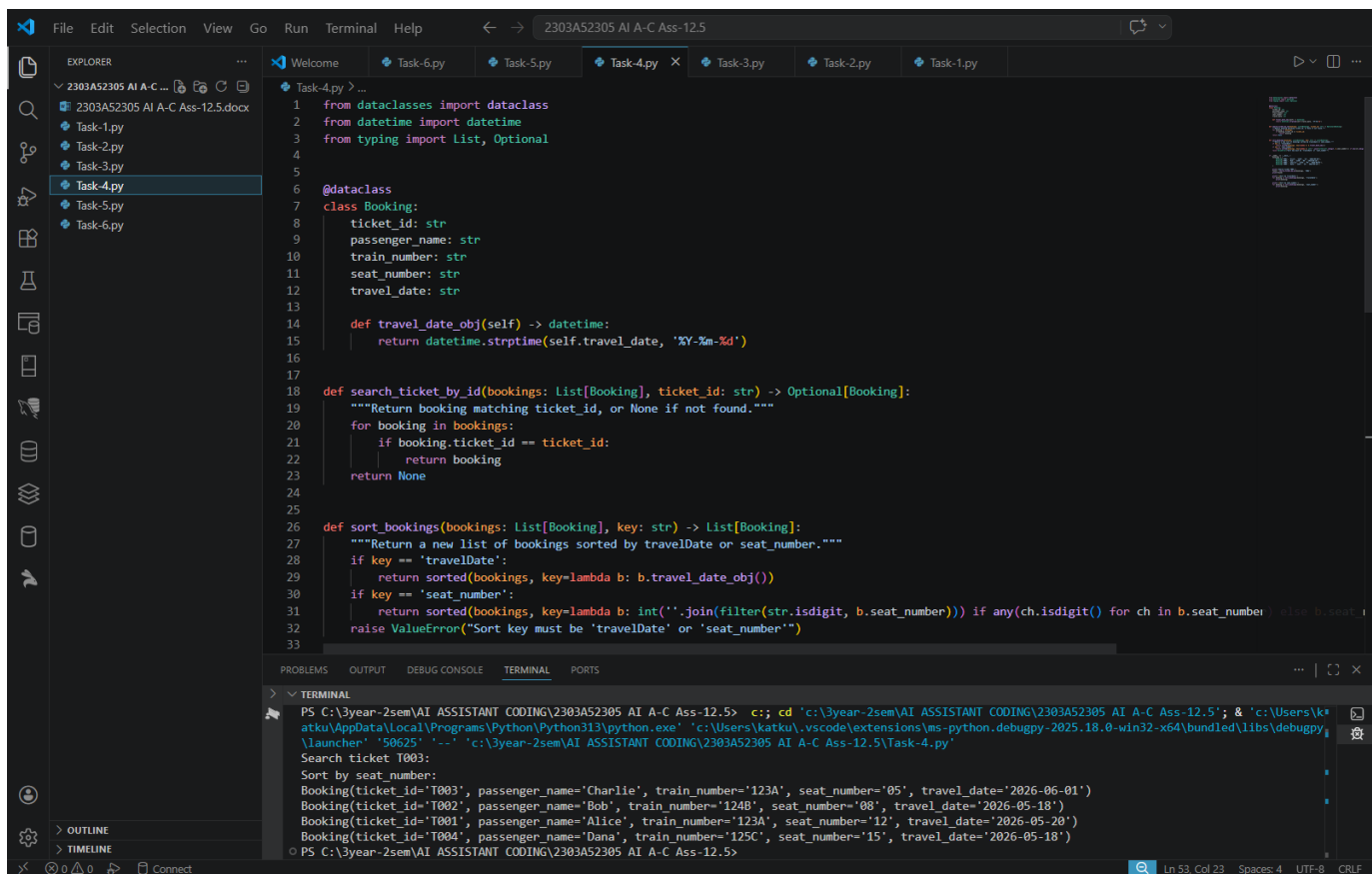
Scenario

A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:

1. Search tickets using ticket ID.
2. Sort bookings based on travel date or seat number.

Student Task

- Identify efficient algorithms using AI assistance.
- Justify the algorithm choices.
- Implement searching and sorting in Python.



```
1 from dataclasses import dataclass
2 from datetime import datetime
3 from typing import List, Optional
4
5
6 @dataclass
7 class Booking:
8     ticket_id: str
9     passenger_name: str
10    train_number: str
11    seat_number: str
12    travel_date: str
13
14    def travel_date_obj(self) -> datetime:
15        return datetime.strptime(self.travel_date, '%Y-%m-%d')
16
17
18 def search_ticket_by_id(bookings: List[Booking], ticket_id: str) -> Optional[Booking]:
19     """Return booking matching ticket_id, or None if not found."""
20     for booking in bookings:
21         if booking.ticket_id == ticket_id:
22             return booking
23     return None
24
25
26 def sort_bookings(bookings: List[Booking], key: str) -> List[Booking]:
27     """Return a new list of bookings sorted by travelDate or seat_number."""
28     if key == 'travelDate':
29         return sorted(bookings, key=lambda b: b.travel_date_obj())
30     if key == 'seat_number':
31         return sorted(bookings, key=lambda b: int(''.join(filter(str.isdigit, b.seat_number))) if any(ch.isdigit() for ch in b.seat_number) else b.seat_number)
32     raise ValueError("Sort key must be 'travelDate' or 'seat_number'")
33
```

PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-12.5> c:\cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-12.5'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '50625' '-...' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-12.5\Task-4.py'

Search ticket T003:

Sort by seat number:

Booking(ticket_id='T003', passenger_name='Charlie', train_number='123A', seat_number='05', travel_date='2026-06-01')

Booking(ticket_id='T002', passenger_name='Bob', train_number='124B', seat_number='08', travel_date='2026-05-18')

Booking(ticket_id='T001', passenger_name='Alice', train_number='123A', seat_number='12', travel_date='2026-05-20')

Booking(ticket_id='T004', passenger_name='Dana', train_number='125C', seat_number='15', travel_date='2026-05-18')

PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-12.5>

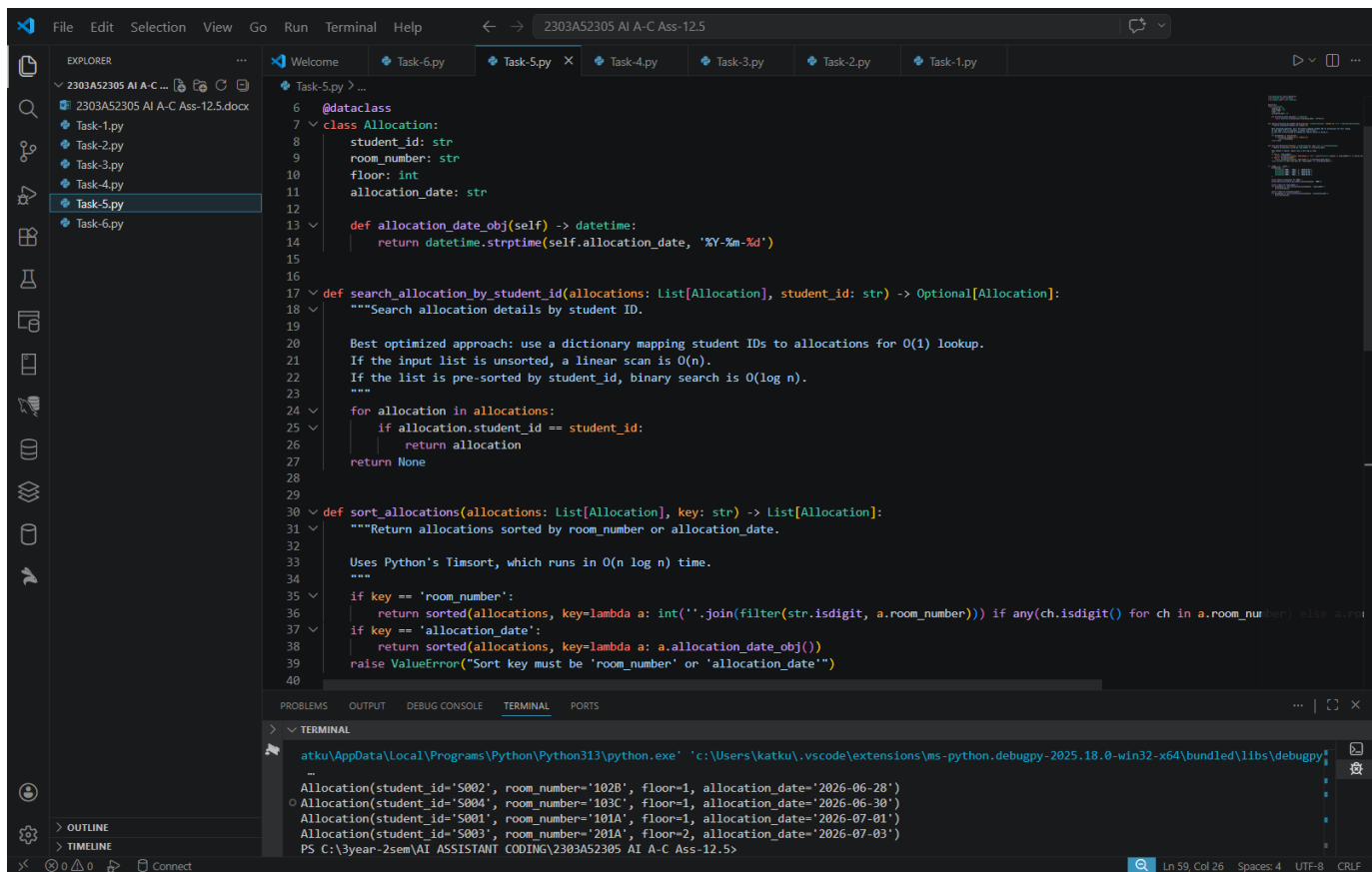
Task Description #5: Smart Hostel Room Allocation System

A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:

1. Search allocation details using student ID.
2. Sort records based on room number or allocation date.

Student Task

- Use AI to suggest optimized algorithms.
- Justify the selections.
- Implement the solution in Python.



```
6 @dataclass
7 class Allocation:
8     student_id: str
9     room_number: str
10    floor: int
11    allocation_date: str
12
13    def allocation_date_obj(self) -> datetime:
14        return datetime.strptime(self.allocation_date, '%Y-%m-%d')
15
16
17    def search_allocation_by_student_id(allocations: List[Allocation], student_id: str) -> Optional[Allocation]:
18        """Search allocation details by student ID.
19
20        Best optimized approach: use a dictionary mapping student IDs to allocations for O(1) lookup.
21        If the input list is unsorted, a linear scan is O(n).
22        If the list is pre-sorted by student_id, binary search is O(log n).
23        """
24        for allocation in allocations:
25            if allocation.student_id == student_id:
26                return allocation
27        return None
28
29
30    def sort_allocations(allocations: List[Allocation], key: str) -> List[Allocation]:
31        """Return allocations sorted by room_number or allocation_date.
32
33        Uses Python's Timsort, which runs in O(n log n) time.
34        """
35        if key == 'room_number':
36            return sorted(allocations, key=lambda a: int(''.join(filter(str.isdigit, a.room_number))) if any(ch.isdigit() for ch in a.room_number) else a.room_number)
37        if key == 'allocation_date':
38            return sorted(allocations, key=lambda a: a.allocation_date_obj())
39        raise ValueError("Sort key must be 'room_number' or 'allocation_date'")
40
```

```
atku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy
-
Allocation(student_id='S802', room_number='102B', floor=1, allocation_date='2026-06-28')
Allocation(student_id='S804', room_number='103C', floor=1, allocation_date='2026-06-30')
Allocation(student_id='S801', room_number='101A', floor=1, allocation_date='2026-07-01')
Allocation(student_id='S803', room_number='201A', floor=2, allocation_date='2026-07-03')
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-12.5>
```

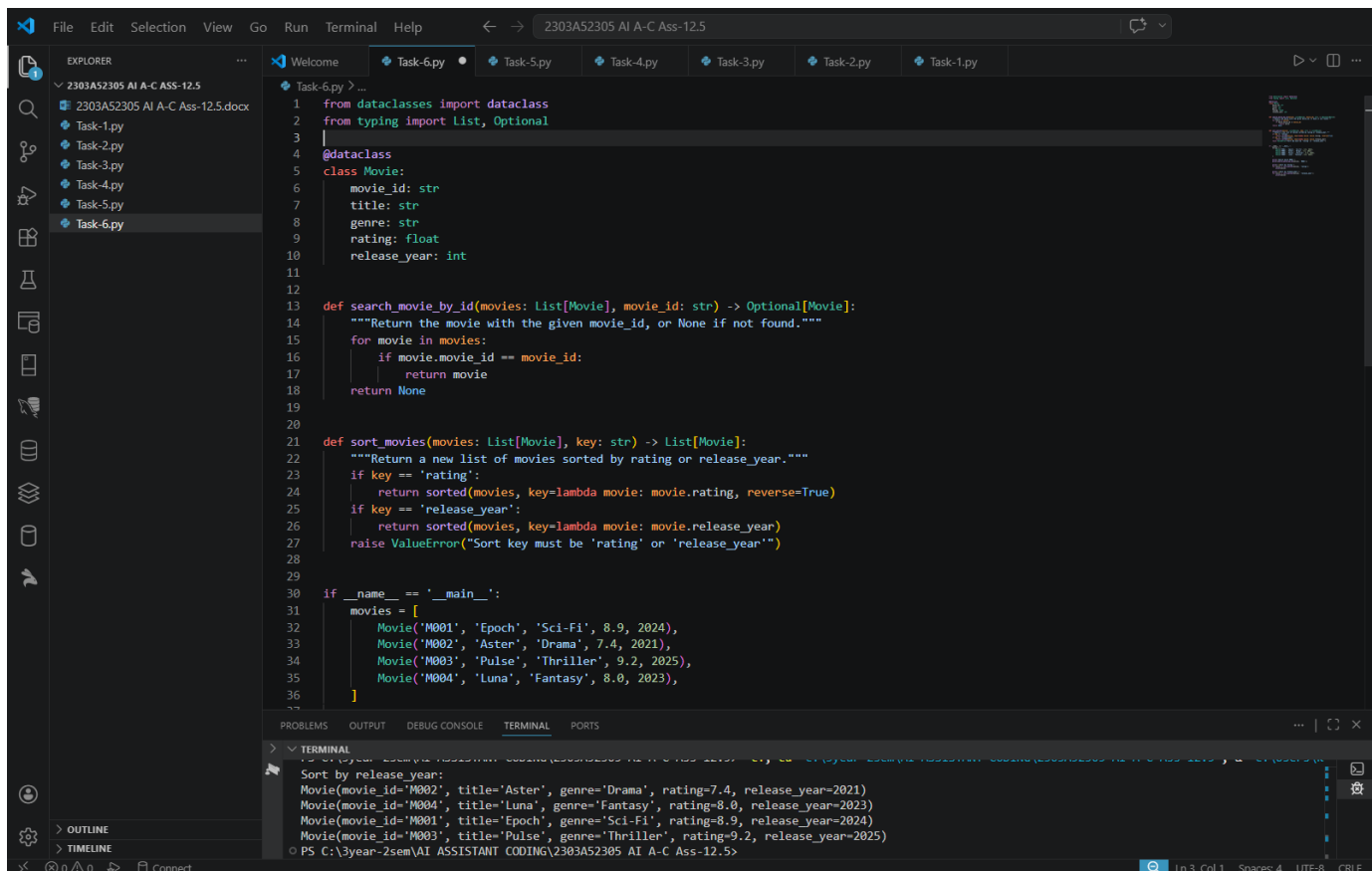
Task Description #6: Online Movie Streaming Platform

A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.
2. Sort movies based on rating or release year.

Student Task

- Recommend searching and sorting algorithms using AI.
- Justify the chosen algorithms.
- Implement Python functions.



```
File Edit Selection View Go Run Terminal Help
2303AS2305 AI A-C Ass-12.5

EXPLORER
2303AS2305 AI A-C Ass-12.5
2303AS2305 AI A-C Ass-12.5.docx
Task-1.py
Task-2.py
Task-3.py
Task-4.py
Task-5.py
Task-6.py

Task-6.py > ...
1 from dataclasses import dataclass
2 from typing import List, Optional
3
4 @dataclass
5 class Movie:
6     movie_id: str
7     title: str
8     genre: str
9     rating: float
10    release_year: int
11
12
13 def search_movie_by_id(movies: List[Movie], movie_id: str) -> Optional[Movie]:
14     """Return the movie with the given movie_id, or None if not found."""
15     for movie in movies:
16         if movie.movie_id == movie_id:
17             return movie
18     return None
19
20
21 def sort_movies(movies: List[Movie], key: str) -> List[Movie]:
22     """Return a new list of movies sorted by rating or release_year."""
23     if key == 'rating':
24         return sorted(movies, key=lambda movie: movie.rating, reverse=True)
25     if key == 'release_year':
26         return sorted(movies, key=lambda movie: movie.release_year)
27     raise ValueError("Sort key must be 'rating' or 'release_year'")
28
29
30 if __name__ == '__main__':
31     movies = [
32         Movie('M001', 'Epoch', 'Sci-Fi', 8.9, 2024),
33         Movie('M002', 'Aster', 'Drama', 7.4, 2021),
34         Movie('M003', 'Pulse', 'Thriller', 9.2, 2025),
35         Movie('M004', 'Luna', 'Fantasy', 8.0, 2023),
36     ]
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

TERMINAL
PS C:\3year-2sem\AI ASSISTANT CODING\2303AS2305 AI A-C Ass-12.5>
Sort by release_year:
Movie(movie_id='M002', title='Aster', genre='Drama', rating=7.4, release_year=2021)
Movie(movie_id='M004', title='Luna', genre='Fantasy', rating=8.0, release_year=2023)
Movie(movie_id='M001', title='Epoch', genre='Sci-Fi', rating=8.9, release_year=2024)
Movie(movie_id='M003', title='Pulse', genre='Thriller', rating=9.2, release_year=2025)
PS C:\3year-2sem\AI ASSISTANT CODING\2303AS2305 AI A-C Ass-12.5>
```